DIPLOMADO EN MACHINE LEARNING Y DATA SCIENCE AVANZADO · DEEP LEARNING

Tarea 4 · Transformers

Clasificación de noticias de Reuters con un codificador Transformer



Guía de estudio: el problema, la teoría de fondo, el código explicado y la intervención

Módulo 5, unidad 4 · Universidad Nacional de Colombia, Facultad de Ingeniería

Notebook: `v2_M5U4_Tarea_4.ipynb`

Natalia Patricia Remolina Rodríguez · 7 de noviembre de 2026

✓ Calificación en UNCode: 100 %, 12 de 12 pruebas en 100

La tarea en una página

El problema

- Asignar cada noticia de la agencia Reuters a uno de **46 temas**.
- Las noticias llegan como listas de índices de palabras, **de longitud variable**.
- 8.982 noticias para entrenar y 2.246 para validar.

La solución

- Un **codificador Transformer** construido pieza por pieza en Keras.
- Un token artificial `[CLS]` al inicio acumula, por atención, el sentido de toda la noticia.
- Solo su fila pasa a una cabeza densa con *softmax* de 46 clases.

La intervención

- `seq_preprocess` pasó de **50 a 100** en dos pruebas.
- La referencia conserva el **final** de las noticias largas, no el inicio.
- El `[CLS]` debe anteponerse **después** de recortar, o se pierde.

El resultado

- Las 12 pruebas en 100 y envío oficial con **100 %**.
- **67,9 %** de accuracy de validación, frente a 68,1 % de la referencia.
- Casi el doble de la línea base de predecir siempre el tema más frecuente, cerca de 36 %.

CÓMO LEER ESTA GUÍA

La guía avanza de lo general a lo concreto:

- La **parte I** plantea el problema.
- La **parte II** explica la teoría necesaria, con ejemplos numéricos pequeños.
- La **parte III** recorre el notebook celda por celda y sigue la forma de los tensores.
- La **parte IV** cuenta qué falló, cómo se diagnosticó y qué se cambió.
- La **parte V** interpreta los resultados.
- La **parte VI** cierra con preguntas de autoevaluación y un glosario.

Las cajas de colores tienen siempre el mismo sentido:

QUÉ SE PEDÍA
la instrucción del enunciado.

INTUICIÓN
la idea detrás del código.

INTERVENCIÓN
lo que hubo que cambiar para llegar al 100 %.

OJO
un detalle fácil de pasar por alto.

Contenido

Parte I · El problema

1. Qué pide la tarea
2. Cómo califica UNCode

Parte II · La teoría de fondo

3. De noticias a números
4. Embeddings de token y de posición
5. La atención por dentro
6. El bloque codificador
7. De vectores a un tema

Parte III · La solución celda por celda

8. Preparación y datos
9. Punto 1 · seq_preprocess
10. Punto 2 · embedding
11. Punto 3 · codificador
12. Punto 4 · get_model
13. Punto 5 · train_model
14. Envío

Parte IV · La intervención

15. El método de diagnóstico
16. El lado del recorte
17. Lo que no hizo falta
18. Lecciones

Parte V · Resultados

19. Las pruebas
20. Curvas de entrenamiento
21. La otra versión
22. Mejoras posibles

Parte VI · Para consolidar

23. Preguntas de autoevaluación
24. Glosario
25. Conclusiones

PARTE I

El problema

Qué hay que resolver, con qué datos y bajo qué reglas de calificación.

1. Qué pide la tarea

Es un problema de **clasificación de texto multiclase**: dado el contenido de una noticia de Reuters, decidir a cuál de 46 temas pertenece, como granos, petróleo, tasas de interés o adquisiciones. La condición central es que el clasificador sea un Transformer **construido pieza por pieza**: nada de modelos preentrenados; el embedding, el bloque codificador y el ensamblaje se programan como capas propias de Keras.

Aspecto	Detalle
Noticias	11.228 etiquetadas, divididas por Keras en 8.982 de entrenamiento y 2.246 de validación
Formato	Cada noticia es una lista de enteros de longitud variable; cada entero es el índice de una palabra, ordenada por frecuencia en el corpus
Vocabulario	Las 20.000 palabras más frecuentes
Etiqueta	Un entero entre 0 y 45. El tema más frecuente, *earn*, ronda el 36 % de las noticias

Por qué un Transformer y no una bolsa de palabras

Un modelo de bolsa de palabras cuenta cuántas veces aparece cada palabra y pierde el contexto. El Transformer, en cambio, construye para cada palabra una representación que depende de las demás: *interest* junto a *rates* no significa lo mismo que en *public interest*. Esa capacidad de leer en contexto es lo que la tarea quiere poner en práctica.

Los cinco puntos calificados

Punto	Función	Qué debe hacer	Entra → sale
1	`seq_preprocess`	Llevar todas las noticias a 200 posiciones con el `[CLS]` al inicio y el relleno al final	listas → `(n, 200)`
2	`get_EmbeddingLayer`	Capa que suma el embedding de cada palabra y el de su posición	`(n, 200)` → `(n, 200, d)`
3	`get_EncoderLayer`	Bloque codificador con atención, red *feed-forward*, residuales, normalización y dropout	`(n, 200, d)` → igual
4	`get_model`	Ensamblar, tomar la salida del `[CLS]` y compilar para 46 clases	`(n, 200)` → `(n, 46)`
5	`train_model`	Preprocesar los datos crudos y entrenar validando cada época	modelo → `history`

2. Cómo califica UNCode

Hay 12 pruebas, de dos a tres por punto, y un envío final. El calificador ejecuta cada función con sus propias entradas y compara contra una solución de referencia:

Nivel	Qué compara	Consecuencia práctica
Forma	Las dimensiones, por ejemplo `(8982, 200)`	Contar mal el `[CLS]` rompe la forma
Contenido exacto	Los enteros de cada fila preprocesada y el número de parámetros	Una forma correcta con el contenido equivocado da 50
Entrenamiento	Que el modelo entrene y alcance la exactitud esperada, con márgenes	Diferencias de milésimas por versión de Keras se toleran

> **OJO**
> El segundo nivel explica toda la intervención de la parte IV: la forma de `seq_preprocess` era correcta desde el principio, pero el contenido de las noticias largas no coincidía.

PARTE II

La teoría de fondo

Lo mínimo que hay que entender para que cada línea del código tenga sentido.

3. De noticias a números

Índices de palabras y los ids reservados

Keras entrega cada noticia ya convertida en índices: el índice 1 es la palabra más común del corpus, el 2 la segunda, y así. Con `index_from=2`, todos los índices se corren dos lugares hacia arriba, y quedan libres dos ids bajos:

Id	Uso	Por qué está libre
0	Relleno	Ninguna palabra lo usa
1	Token `[CLS]`	Lo libera `index_from=2`; por eso `cls_id = 1`
≥ 2	Palabras	Índice original + 2

Longitud fija: relleno y recorte

Una red necesita entradas del mismo tamaño para apilarlas en una matriz. Aquí todo se lleva a 200 posiciones, contando el `[CLS]`, así que quedan 199 para palabras:

Noticia corta de 80 palabras

`[CLS]=1` `w1` `w2` `…` `w80` `0` `0` `…` `0` · 119 ceros al final

Noticia larga de 500 palabras

`w1 … w301 descartadas` `[CLS]=1` `w302` `w303` `…` `w500` · se conserva el final

Figura 1. Las dos reglas de `seq_preprocess`: las noticias cortas se rellenan al final y a las largas se les descarta el inicio.

> **INTUICIÓN · POR QUÉ CONSERVAR EL FINAL**
>
> Los cables de Reuters suelen cerrar con las cifras y la síntesis del tema: «… net profit rose to 10 mln dlrs from six mln dlrs». Para decidir el tema, el final es más informativo que el encabezado.

4. Embeddings de token y de posición

Un entero como 417 no dice nada del significado. Un **embedding** es una tabla entrenable que asigna a cada id un vector, aquí de 16 números. Al principio son aleatorios; al entrenar se acomodan para que palabras con funciones parecidas queden cerca.

La atención no distingue el orden: para ella una noticia es un conjunto. Por eso se suma un segundo embedding que aprende un vector por cada **posición**, de 0 a 199:

```
x[i] = token_emb(id del token i) + pos_emb(i)
```

El primer sumando dice «qué palabra es» y el segundo «dónde está». En esta tarea el embedding de posición es **aprendido**, a diferencia de las funciones seno y coseno fijas del artículo original.

Tabla	Tamaño	Parámetros	Qué guarda
`token_emb`	20.000 × 16	320.000	Un vector por palabra
`pos_emb`	200 × 16	3.200	Un vector por posición

5. La atención por dentro

5.1 Consultas, claves y valores

Cada vector de entrada se proyecta en tres vectores con matrices entrenables:

- **Q, consulta:** lo que el token busca.
- **K, clave:** lo que cada token ofrece para ser encontrado.
- **V, valor:** la información que entrega si lo eligen.

$$\text{Atención}(Q, K, V) = \text{softmax}\left(Q \cdot K^{\intercal} / \sqrt{d} \right) \cdot V$$

1. `Q·Kᵀ` compara cada consulta con cada clave y da una matriz de puntajes.
2. Dividir por `√d` evita puntajes enormes que saturarían el *softmax*.
3. `softmax` convierte cada fila en pesos positivos que suman 1.
4. Multiplicar por `V` da, para cada token, el promedio ponderado de los valores.

5.2 Un ejemplo con números

Tres tokens de dimensión 2, con Q, K y V iguales a la entrada para simplificar:

Token	X	Puntajes $Q \cdot K^{\intercal}/\sqrt{2}$			Pesos tras softmax			Salida
		a	b	c	a	b	c	
a	[1, 0]	0,71	0	0,71	0,40	0,20	0,40	[0,80, 0,60]
b	[0, 1]	0	0,71	0,71	0,20	0,40	0,40	[0,60, 0,80]
c	[1, 1]	0,71	0,71	1,41	0,25	0,25	0,50	[0,75, 0,75]

El token *a* se parece a sí mismo y a *c*, y por eso les da más peso. Su salida ya no es solo «a»: incorpora algo de los demás. Eso significa que un vector «tenga contexto».

5.3 Varias cabezas y auto-atención

Con `num_heads=2` el cálculo se hace dos veces en paralelo, con matrices propias, y cada cabeza puede especializarse en un tipo de relación. Como aquí la noticia se mira a sí misma, `att(x, x)`, se llama **auto-atención**. A diferencia del decodificador de la Tarea 5, no hay máscara causal: para clasificar, cada palabra puede mirar a todas, antes y después.

5.4 El papel del token `[CLS]`

El `[CLS]` entra al codificador como un vector sin significado propio: solo su embedding y el de la posición 0. En la auto-atención, su consulta compara contra las claves de las 199 palabras, y su salida es una mezcla ponderada de todas ellas. Después del bloque, esa fila es un **resumen aprendido de la noticia**, y es lo único que ve el clasificador.

> **INTUICIÓN · POR QUÉ NO PROMEDIAR LAS 200 SALIDAS**
> Promediar también funcionaría, pero trata igual a todas las palabras. El `[CLS]` deja que el propio modelo aprenda a qué palabras atender para decidir el tema. Es la misma idea de BERT.

6. El bloque codificador

Alrededor de la atención hay piezas que se repiten:

Pieza	Qué hace	Por qué hace falta
Conexión residual	Suma la entrada a la salida del subbloque	Si el subbloque no aporta, la información pasa igual; ayuda al gradiente
Normalización de capa	Reescala cada vector a media 0 y varianza 1, con dos parámetros entrenables	Mantiene los valores en escala estable
Feed-forward	Dos capas densas por posición: 16 → 32 con ReLU → 16	La atención solo mezcla; esta red transforma de forma no lineal
Dropout	Apaga al azar el 10 % de los valores al entrenar	Evita depender de neuronas concretas; en inferencia no actúa

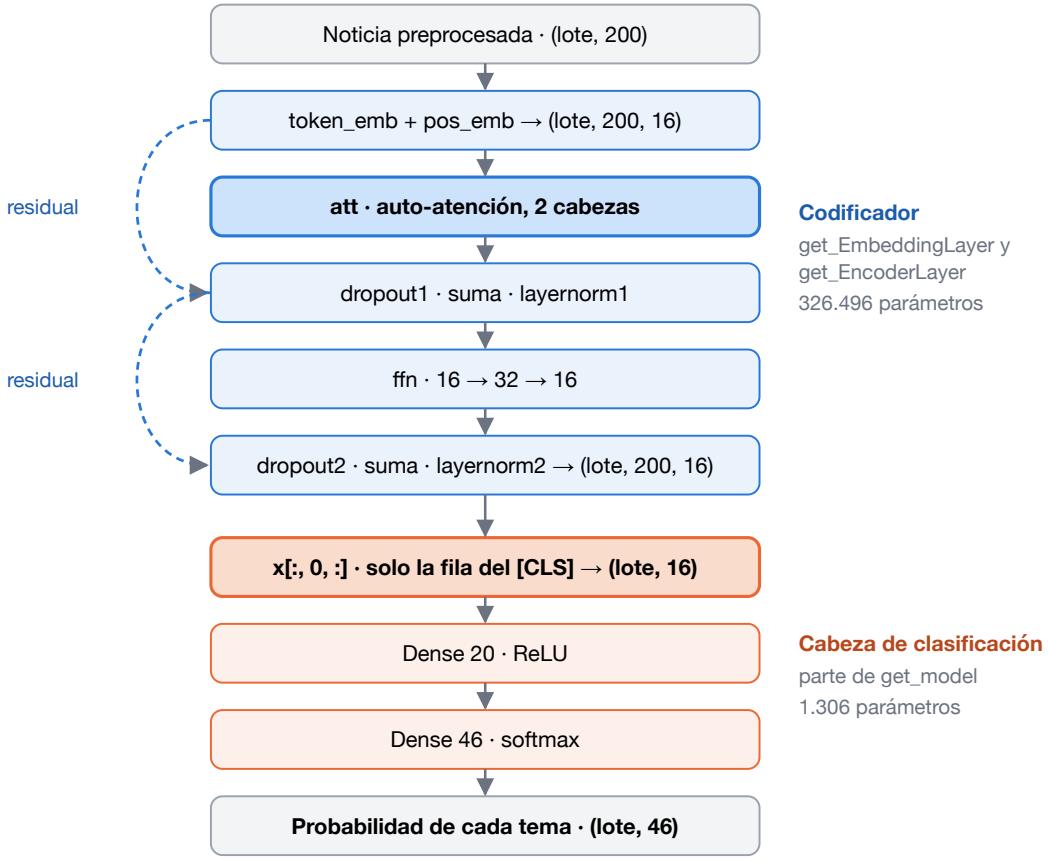


Figura 2. El modelo completo con los nombres de atributos del notebook. Las flechas punteadas son las conexiones residuales. De las 200 salidas del codificador, la cabeza de clasificación solo usa la del `[CLS]`.

7. De vectores a un tema

7.1 Softmax y pérdida dispersa

La última capa produce 46 puntajes y *softmax* los convierte en probabilidades que suman 1. La pérdida es la entropía cruzada, `−log p(tema correcto)`: vale 0 si el modelo da probabilidad 1 al tema correcto y ln 46 ≈ 3,8 si reparte por igual.

Si la etiqueta es…	Ejemplo	Pérdida que corresponde
Un entero	`3`	`sparse_categorical_crossentropy` ✓ este caso
Un vector *one-hot*	`[0, 0, 0, 1, 0, …, 0]`	`categorical_crossentropy`

Las dos calculan lo mismo; la versión dispersa evita construir 46 posiciones por noticia. Es la «forma dispersa» que pide decidir la nota del enunciado.

7.2 Entrenamiento

- **Adam** ajusta cada peso con una tasa propia según la historia de sus gradientes, con tasa base 0,0001.
- Lotes de 32 noticias: 8.982 / 32 da **281 pasos por época**.
- Al final de cada época se mide la validación con noticias que no se usan para ajustar pesos.

PARTE III

La solución, celda por celda

El código tal como quedó, explicado línea por línea y con la forma de los datos en cada paso.

8. Preparación y datos

CELDAS 1 A 4 **Calificador, librerías, semillas y versiones**

CÓDIGO

```
%%capture
!pip install rlxcrypt
!wget --no-cache -O session.pye -q https://raw.githubusercontent.com/JuezUN/INGInious/
                                   master/external%20libs/session.pye

grader = session.LoginSequence("DL-GroupMLDS-5-2026-2@3feead7a-0040-47fc-b701-11492b6ea6c8")

random.seed(0)
keras.utils.set_random_seed(0)
np.random.seed(0)
```

SALIDA

```
Python 3.13.16
NumPy 2.1.3
keras 3.13.2
```

- `rlxcrypt` descifra el módulo del calificador; `LoginSequence` crea el objeto `grader` y pide las credenciales de UNCode.
- Las **semillas** fijan el azar de Python, Keras y NumPy para que los pesos iniciales sean siempre los mismos.
- El entorno es más nuevo que el del enunciado, Python 3.11.12, NumPy 2.0.2 y Keras 3.8.0, y todo pasó igual.

CELDA 5 **Carga de Reuters**

CÓDIGO

```
vocab_size = 20000

(x_train, y_train), (x_val, y_val) = keras.datasets.reuters.load_data(
    num_words=vocab_size, index_from=2, path=0, start_char=None)
words = keras.datasets.reuters.get_word_index(path="reuters_word_index.json")
words = {valor: clave for clave, valor in words.items()}
```

SALIDA

```
8982 Training sequences
2246 Validation sequences
```

- `num_words=20000` deja solo las 20.000 palabras más frecuentes.
- `index_from=2` libera los ids 0 y 1, para el relleno y el `[CLS]`.

- `start_char=None` evita que Keras agregue su propio token de inicio.
- `get_word_index` da el diccionario palabra → índice, y la última línea lo invierte.

CELDA 6 **Ver un ejemplo**

CÓDIGO
```
print(f"texto: {' '.join([words[i] for i in x_train[0]])}")
```

SALIDA
```
texto: of of a not 3 move in by should 22 in rebates dollar 000 reuters four after about
and may in on february said on some reuter after about revs that secretary at and …
```

OJO · EL TEXTO SALE DESORDENADO Y NO ES UN ERROR DE LA SOLUCIÓN

Con `index_from=2`, cada índice de la secuencia es el índice original **más 2**. `get_word_index` devuelve los originales y esta celda de la plantilla no resta ese 2, así que cada palabra impresa es otra que está dos lugares más arriba en el ranking. Para leer la noticia real habría que usar `words.get(i - 2)`. La celda es solo de inspección: el modelo trabaja con los índices tal como vienen.

9. Punto 1 · Normalización de secuencias

QUÉ SE PEDÍA

Que todas las secuencias midan `max_len`, con el `[CLS]` incluido. Dos consideraciones explícitas: el `[CLS]` **cuenta** dentro del tamaño y el relleno va en la parte **final**.

CELDA 7 **Función calificada `seq_preprocess`**

CÓDIGO
```
def seq_preprocess(X, cls_id, max_len):
    # Se rellena/trunca a max_len - 1 (espacio para [CLS]); relleno al final
    X_pad = keras.preprocessing.sequence.pad_sequences(
        X, maxlen=max_len - 1, padding='post', value=0
    )
    cls_col = np.full((X_pad.shape[0], 1), cls_id, dtype=X_pad.dtype)
    X_out = np.concatenate([cls_col, X_pad], axis=1)
    return X_out
```

Línea por línea

- `maxlen=max_len - 1`: 199 posiciones, para dejar el lugar del `[CLS]`.
- `padding='post'`: los ceros van al final, como pide el enunciado.
- **No se indica `truncating`**, así que se usa el valor por defecto de Keras, `'pre'`: en noticias largas se descarta el inicio. Es lo que espera el calificador.
- `value=0`: el relleno usa el id 0.
- `np.full((n, 1), cls_id, dtype=X_pad.dtype)`: una columna de unos, con el mismo tipo de dato que las secuencias.
- `np.concatenate([cls_col, X_pad], axis=1)`: pega la columna al frente. Cada fila queda `[1, w1, …, 0, 0]`.

Ejemplo con `cls_id = 1` y `max_len = 4`

Noticia	Tras recortar o rellenar a 3	Con el `[CLS]`
`[5]`	`[5, 0, 0]`	`[1, 5, 0, 0]`
`[5, 6, 7]`	`[5, 6, 7]`	`[1, 5, 6, 7]`
`[8, 9, 10, 11, 12]`	`[10, 11, 12]`, se descarta el inicio	`[1, 10, 11, 12]`

SALIDA DE LA CELDA 8

```
Tamaño de la particion de entrenamiento:(8982, 200)
Tamaño de la particion de validacion:(2246, 200)
```

Pruebas 1_1, 1_2 y 1_3: 100, 100 y 100, después de la intervención de la parte IV.

10. Punto 2 · Capa de embedding

QUÉ SE PEDÍA

Una capa personalizada que entregue, para cada token, su embedding más la codificación posicional.

CELDA 12 **Función calificada `get_EmbeddingLayer`**

CÓDIGO

```
def get_EmbeddingLayer(max_len, vocab_size, embed_dim):

    class TokenAndPositionEmbedding(keras.layers.Layer):
        def __init__(self, max_len, vocab_size, embed_dim):
            super().__init__()
            self.token_emb = keras.layers.Embedding(input_dim=vocab_size,
                                                    output_dim=embed_dim)
            self.pos_emb = keras.layers.Embedding(input_dim=max_len, output_dim=embed_dim)

        def call(self, x):
            seq_len = keras.ops.shape(x)[-1]
            positions = keras.ops.arange(0, seq_len, 1)
            positions = self.pos_emb(positions)
            x = self.token_emb(x)
            return x + positions

    layer = TokenAndPositionEmbedding(max_len, vocab_size, embed_dim)
    return layer
```

INTUICIÓN · CÓMO SE ESCRIBE UNA CAPA PROPIA

`__init__` **declara** las piezas, que se crean una vez y guardan sus pesos; `call` las **usa** cada vez que entran datos. La función externa solo construye y devuelve la instancia.

Línea de `call`, lote de 10 y `embed_dim = 128`	Forma
`x`, la entrada	`(10, 200)`
`positions = arange(0, 200)`	`(200,)`
`pos_emb(positions)`	`(200, 128)`

`token_emb(x)`	`(10, 200, 128)`
`x + positions`, la suma se repite para las 10 noticias	`(10, 200, 128)`

Se usan operaciones de `keras.ops` y no de NumPy para que funcionen dentro del grafo de cómputo. La plantilla traía esta función con sangría de 2 espacios y se pasó a 4.

SALIDA DE LA CELDA 13

```
Codificación de sequencias de entrenamiento: (10, 200, 128)
```

Pruebas 2_1, 2_2 y 2_3: 100, 100 y 100 en el primer intento.

11. Punto 3 · Codificador

QUÉ SE PEDÍA

Una capa personalizada con el bloque codificador. Recibe `embed_dim`, `num_heads`, `ff_dim` y `rate`.

CELDA 17 **Función calificada `get_EncoderLayer`**

CÓDIGO

```
def get_EncoderLayer(embed_dim, num_heads, ff_dim, rate):
    class TransformerBlock(keras.layers.Layer):
        def __init__(self, embed_dim, num_heads, ff_dim, rate):
            super().__init__()
            self.att = keras.layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
            self.ffn = keras.Sequential([
                keras.layers.Dense(ff_dim, activation="relu"),
                keras.layers.Dense(embed_dim),
            ])
            self.layernorm1 = keras.layers.LayerNormalization(epsilon=1e-6)
            self.layernorm2 = keras.layers.LayerNormalization(epsilon=1e-6)
            self.dropout1 = keras.layers.Dropout(rate)
            self.dropout2 = keras.layers.Dropout(rate)

        def call(self, inputs, training=False):
            attn_output = self.att(inputs, inputs)                  # 1. auto-atención
            attn_output = self.dropout1(attn_output, training=training)
            out1 = self.layernorm1(inputs + attn_output)            # 2. residual y norma
            ffn_output = self.ffn(out1)                             # 3. feed-forward
            ffn_output = self.dropout2(ffn_output, training=training)
            return self.layernorm2(out1 + ffn_output)               # 4. residual y norma

    layer = TransformerBlock(embed_dim, num_heads, ff_dim, rate)
    return layer
```

Paso, con `embed_dim = 16` como en el modelo final	Forma	Qué significa
`inputs`	`(lote, 200, 16)`	Una fila por token
Pesos internos de la atención	`(lote, 2, 200, 200)`	Cada cabeza: cuánto mira cada token a cada otro

`att(inputs, inputs)`	`(lote, 200, 16)`	Cada fila ya mezcla información de las 200
`layernorm1(inputs + attn_output)`	`(lote, 200, 16)`	Residual y normalización
`ffn(out1)`	`(lote, 200, 32)` → `(lote, 200, 16)`	Transformación por posición
`layernorm2(out1 + ffn_output)`	`(lote, 200, 16)`	Misma forma que la entrada

- `epsilon=1e-6` evita divisiones por cero en la normalización.
- **Sobre `training`:** la plantilla definía `call(self, inputs)`. Aquí se recibe `training` y se pasa a los dropout; durante `fit` Keras lo llama con `True`.
- Que la salida tenga la forma de la entrada permitiría apilar varios bloques en serie.

SALIDA DE LA CELDA 18

```
Codificación de sequencias de entrenamiento: (10, 200, 128)
```

Pruebas 3_1 y 3_2: 100 y 100 en el primer intento.

12. Punto 4 · Construir el modelo

QUÉ SE PEDÍA

Ensamblar las piezas, usar el `[CLS]` como representación final y emplear activación, pérdida y métrica adecuadas para clasificación multiclase. Una nota pide decidir si usar la forma dispersa de la pérdida.

CELDA 21 **Función calificada `get_model`**

CÓDIGO

```
def get_model(max_len, vocab_size, embed_dim, num_heads, ff_dim, rate, num_clases,
              learning_rate, n_neurons):
    np.random.seed(0)
    keras.utils.set_random_seed(0)
    keras.backend.clear_session()

    inputs = keras.layers.Input(shape=(max_len,))
    x = get_EmbeddingLayer(max_len, vocab_size, embed_dim)(inputs)
    x = get_EncoderLayer(embed_dim, num_heads, ff_dim, rate)(x)
    x = x[:, 0, :]  # representación del token [CLS]
    x = keras.layers.Dense(n_neurons, activation="relu")(x)
    outputs = keras.layers.Dense(num_clases, activation="softmax")(x)

    model = keras.Model(inputs=inputs, outputs=outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

- Las semillas y `clear_session()` venían dadas: el modelo sale idéntico en cada llamada, incluidos los nombres `dense_2` y `dense_3`.

- `x = x[:, 0, :]` **es la línea más importante:** de las 200 filas toma solo la del `[CLS]`. En el resumen aparece como la capa `get_item`, con salida `(None, 16)` y 0 parámetros.
- `Dense(20, relu)` y `Dense(46, softmax)` forman la cabeza de clasificación.
- `sparse_categorical_crossentropy` porque las etiquetas son enteros; Adam con la tasa recibida; métrica `accuracy`.

Resumen del modelo y de dónde sale cada parámetro

Componente	Cálculo	Parámetros
Embedding de palabras	20.000 × 16	320.000
Embedding de posiciones	200 × 16	3.200
Atención: Q, K y V	3 × 544, con 16 × 2 × 16 pesos y 32 sesgos cada una	1.632
Atención: proyección de salida	2 × 16 × 16 + 16	528
Feed-forward	16 × 32 + 32 + 32 × 16 + 16	1.072
Dos normalizaciones	2 × 32	64
`dense_2`	16 × 20 + 20	340
`dense_3`	20 × 46 + 46	966
Total · 1,25 MB, idéntico a la salida esperada		**327.802**

El 98,6 % del modelo está en los embeddings. El bloque Transformer, que es el que lee en contexto, tiene apenas 3.296 parámetros.

CELDA 23 `pprint.pprint(model.get_config())`

> **OJO · DIFERENCIA CON LA SALIDA ESPERADA**
>
> La salida esperada muestra, dentro de las capas propias, sus hiperparámetros: `embed_dim`, `max_len`, `vocab_size`, `num_heads`, `ff_dim` y `rate`. Para eso hace falta implementar `get_config()`. Esta versión no lo implementa y las pruebas 4_1 y 4_2 dieron 100 de todos modos. La salida esperada, además, dice `embed_dim: 32` aunque el modelo se construye con 16: viene de otra ejecución.

Pruebas 4_1 y 4_2: 100 y 100 en el primer intento.

13. Punto 5 · Entrenamiento

> **QUÉ SE PEDÍA**
>
> Recibir el modelo compilado y las secuencias **crudas**, preprocesarlas, entrenar y devolver `history`.

CELDA 26 **Función calificada** `train_model`

CÓDIGO

```
def train_model(model, cls_id, X_train, Y_train, X_val, Y_val, max_len, epochs):
    X_train_p = seq_preprocess(X_train, cls_id, max_len)
    X_val_p = seq_preprocess(X_val, cls_id, max_len)
```

```
    history = model.fit(
        X_train_p, Y_train,
        validation_data=(X_val_p, Y_val),
        epochs=epochs,
    )
    return history
```

- Entrenamiento y validación pasan por el **mismo** `seq_preprocess`: el modelo debe ver datos con el mismo formato en ambos.
- Sin `batch_size`, Keras usa 32: los 281 pasos por época de la salida esperada.
- `validation_data` mide la validación al final de cada época.

OJO · ERRATA DEL ENUNCIADO

Dice que la función devuelve «el modelo y `history`», pero la celda de prueba recibe un solo valor. El modelo se entrena en el lugar, así que basta devolver `history`.

SALIDA DE LA CELDA 27 · PRIMERA Y ÚLTIMA ÉPOCA

```
Epoch 1/10
281/281 ━━━━━━━━━━━━━━━━━━━━ 19s 36ms/step - accuracy: 0.3195 - loss: 2.9635 - val_accuracy:
0.4920
                                        - val_loss: 2.5244
Epoch 10/10
281/281 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.7341 - loss: 1.0972 - val_accuracy:
0.6794
                                       - val_loss: 1.3299
```

Pruebas 5_1 y 5_2: 100 y 100. Son las más lentas: cada una construye un modelo y lo entrena desde cero con estas funciones.

14. Envío

CELDA 30 `grader.submit_task(globals())`

SALIDA

```
TEST Test 1_1 , Your test grade: 100      TEST Test 1_2 , Your test grade: 100
TEST Test 1_3 , Your test grade: 100      TEST Test 2_1 , Your test grade: 100
TEST Test 2_2 , Your test grade: 100      TEST Test 2_3 , Your test grade: 100
TEST Test 3_1 , Your test grade: 100      TEST Test 3_2 , Your test grade: 100
TEST Test 4_1 , Your test grade: 100      TEST Test 4_2 , Your test grade: 100
TEST Test 5_1 , Your test grade: 100      TEST Test 5_2 , Your test grade: 100
Your submission grade: 100.0%
```

Hubo un envío anterior con 1_2 y 1_3 en 50, hecho antes de corregir el recorte.

PARTE IV

La intervención: cómo se llegó al 100 %

Qué falló, cómo se averiguó la causa y qué se cambió exactamente.

15. El método de diagnóstico

1. Ejecutar	→	2. Leer	→	3. Plantear	→	4. Probar	→	5. Repetir
la prueba del calificador		esperado frente a obtenido		una hipótesis concreta		cambiando una sola cosa		las pruebas que dependen de ello

La nota parcial ya es una pista: un 100 en la prueba de forma y un 50 en las de contenido dicen que la estructura está bien y que algo cambia en una parte de los datos.

16. Intervención · el lado del recorte

PROBLEMA

`seq_preprocess` obtuvo **100 en 1_1 y 50 en 1_2 y 1_3**. La forma era correcta.

DIAGNÓSTICO

1. **1_1 dio 100:** verifica la forma y las noticias cortas, en las que no se recorta nada.
2. **1_2 y 1_3 dieron 50:** su retroalimentación mostraba el arreglo esperado frente al obtenido. En las noticias largas, el esperado terminaba en índices distintos, y sus primeras palabras después del `[CLS]` eran de la mitad de la noticia.
3. **Conclusión:** la referencia conserva el **final**, el recorte `'pre'`, que además es el valor por defecto de Keras. El enunciado solo decía de qué lado va el relleno.

ANTES · 50, CONSERVA EL INICIO

```
def seq_preprocess(X, cls_id, max_len):
    X_cls = [[cls_id] + list(x) for x in X]                  # 1. antepone el [CLS]
    X_out = keras.utils.pad_sequences(X_cls, maxlen=max_len,  # 2. recorta por el final
                                      padding='post', truncating='post', value=0)
    return np.array(X_out)
```

DESPUÉS · 100, CONSERVA EL FINAL

```
def seq_preprocess(X, cls_id, max_len):
    X_pad = keras.preprocessing.sequence.pad_sequences(       # 1. recorta por el inicio
        X, maxlen=max_len - 1, padding='post', value=0
    )
    cls_col = np.full((X_pad.shape[0], 1), cls_id, dtype=X_pad.dtype)
    X_out = np.concatenate([cls_col, X_pad], axis=1)          # 2. antepone el [CLS]
    return X_out
```

Por qué no bastaba cambiar `'post'` por `'pre'`

Las tres variantes con la noticia `[8, 9, 10, 11, 12]`, `cls_id = 1` y `max_len = 4`:

Variante	Resultado	Problema
Primera versión: `[CLS]` antes y recorte `'post'`	`[1, 8, 9, 10]`	Conserva el inicio: no coincide con la referencia
Solo cambiar a `'pre'`, con el `[CLS]` antes	`[9, 10, 11, 12]`	**El `[CLS]` desaparece**: el clasificador leería una palabra cualquiera
Versión final: recortar a 3 y anteponer después	`[1, 10, 11, 12]`	Ninguno ✓

POR QUÉ FUNCIONA

Recortar y rellenar a `max_len − 1` deja exactamente un lugar libre, y el `[CLS]` se pega después, así que nunca puede perderse. Tras el cambio se repitieron las pruebas 1_x, que pasaron a 100, y también 5_1 y 5_2, porque `train_model` usa `seq_preprocess`. El notebook guarda la huella del proceso: los contadores de ejecución saltan del 6 al 11 justo en esta función.

17. Lo que no hizo falta

Elemento	Por qué parecía necesario	Qué pasó
`get_config()` en las capas	La salida esperada de `model.get_config()` muestra los hiperparámetros de las capas propias	Las pruebas 4_1 y 4_2 dieron 100 sin implementarlo
`batch_size=32` explícito	La salida esperada muestra 281 pasos por época	32 es el valor por defecto de `fit`
`truncating='pre'` explícito	Es la clave de la corrección	Es el valor por defecto de `pad_sequences`; escribirlo solo hace el código más claro

18. Lecciones

LECCIONES

1. **Qué extremo se recorta es una decisión de diseño** con consecuencias: aquí valía 50 puntos, y además tiene sentido en noticias de agencia.
2. **El orden de las operaciones importa:** anteponer y luego recortar no es lo mismo que recortar y luego anteponer.
3. **La retroalimentación de esperado frente a obtenido contiene la pista exacta.** Leerla evita probar a ciegas.
4. **Los valores por defecto son decisiones**, aunque nadie los escriba: `truncating='pre'` y `batch_size=32`.
5. **Después de corregir una función, hay que repetir las pruebas de todo lo que la usa.**

PARTE V

Resultados y análisis

Cuánto aprendió el modelo, cómo se compara con la referencia y qué lo limita.

19. Las pruebas

Punto	Pruebas	Primera versión	Versión final
1 · `seq_preprocess`	1_1, 1_2, 1_3	100, 50, 50	**100, 100, 100**
2 · `get_EmbeddingLayer`	2_1, 2_2, 2_3	100, 100, 100	**100, 100, 100**
3 · `get_EncoderLayer`	3_1, 3_2	100, 100	**100, 100**
4 · `get_model`	4_1, 4_2	100, 100	**100, 100**
5 · `train_model`	5_1, 5_2	100, 100	**100, 100**
Envío oficial con `submit_task`			**100 %**

20. Curvas de entrenamiento

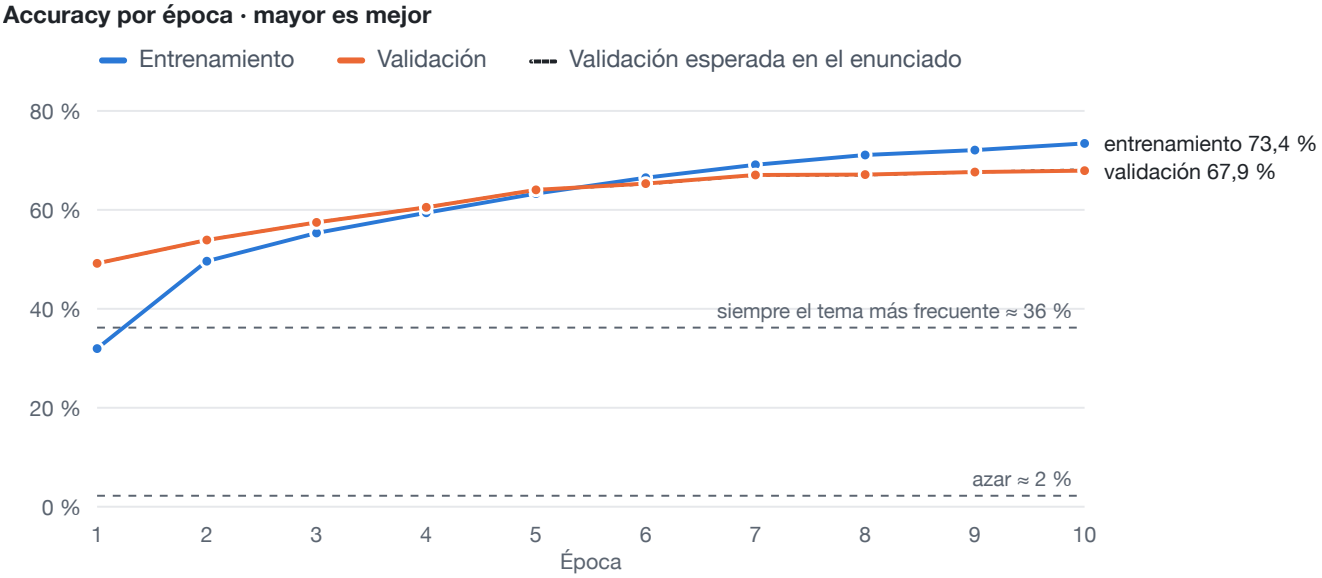


Figura 3. Accuracy de entrenamiento y de validación. La línea punteada fina es la validación esperada en el enunciado: coincide con la obtenida hasta la época 5. Las líneas grises de trazos son las dos líneas base.

20.1 El modelo aprendió de forma sostenida

La accuracy de entrenamiento subió de 32 % a 73,4 % y la de validación, de 49 % a 67,9 %. Las dos suben juntas durante las 10 épocas y la brecha final es de unos 5,5 puntos: no hay señales fuertes de sobreajuste.

20.2 Cómo dimensionar el 67,9 %

Referencia	Accuracy
Adivinar al azar entre 46 temas	≈ 2 %
Predecir siempre el tema más frecuente	≈ 36 %
Este modelo, época 10	**67,9 %**
Referencia del enunciado, época 10	68,1 %

El modelo casi duplica la línea base de la clase mayoritaria: la atención extrae señal temática real.

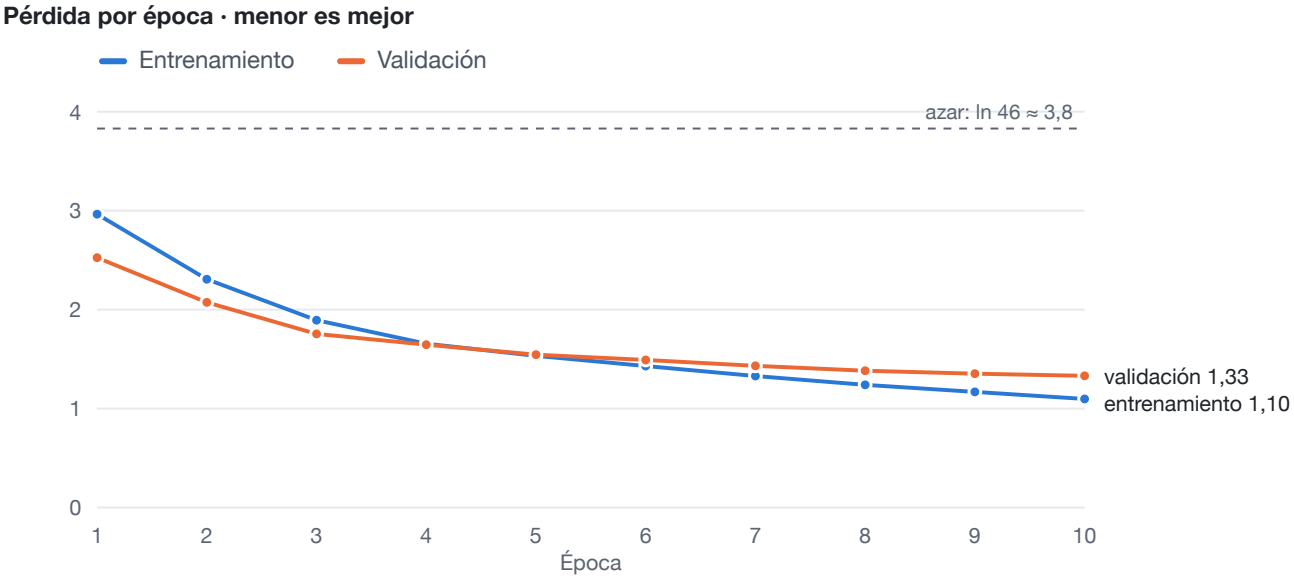


Figura 4. Pérdida de entrenamiento y de validación. La línea de trazos es la pérdida de repartir la probabilidad por igual entre 46 temas.

20.3 Aún había margen

La pérdida de validación seguía bajando en la época 10, de 1,3534 a 1,3299. Con más épocas, un embedding mayor o más bloques probablemente mejoraría, pero las pruebas fijan los hiperparámetros.

20.4 Coincidencia con la referencia

Las cinco primeras épocas de validación son **idénticas** a la salida esperada, en accuracy y en pérdida, hasta el cuarto decimal. Eso indica que la arquitectura, el orden de creación de las capas y la inicialización son los mismos de la referencia. Desde la época 6 aparecen diferencias de milésimas, atribuibles al no determinismo de la GPU y a Keras 3.13 frente a 3.8.

> **OJO · LA ACCURACY DE ENTRENAMIENTO NO COINCIDE AUNQUE LA VALIDACIÓN SÍ**
>
> En la época 1 da 0,3195 frente a 0,1947 esperado. Si la validación es idéntica, los pesos al final de la época son los mismos, así que la diferencia no está en el modelo. Lo más probable es que cada versión de Keras promedie de forma distinta la métrica que muestra mientras avanza la época.

21. La otra versión de la solución

El análisis original en PDF describe una ejecución distinta de la archivada. Las dos obtienen 12 de 12 y 100 %.

Aspecto	Versión archivada	Versión del análisis
`seq_preprocess`	Recorte por defecto y `np.concatenate`	`truncating='pre'` explícito y `np.hstack`; el resultado es el mismo
Capas propias	Sin `get_config()`	Con `get_config()` y `**kwargs`
`call` del codificador	`training=False`	`training=None`
Época 10	acc 0,7341 · val_acc 0,6794 · val_loss 1,3299	acc 0,7383 · val_acc 0,6861 · val_loss 1,3119

22. Mejoras posibles

Ninguna era necesaria para la nota; sirven para entender qué limita al modelo.

1. Que la atención ignore el relleno

En una noticia de 80 palabras, 119 de las 200 posiciones son ceros, y la atención del `[CLS]` también las mira, porque el modelo no recibe ninguna máscara. Pasar a la atención una máscara que excluya los ceros haría que el resumen dependa solo de palabras reales:

IDEA

```
mascara = keras.ops.not_equal(ids, 0)                # True en palabras reales
attn_output = self.att(x, x, attention_mask=mascara[:, None, :])
```

Cambiaría los resultados, así que no coincidiría con la referencia del calificador.

2. Que las capas sean serializables

CÓDIGO

```
def get_config(self):
    config = super().get_config()
    config.update({"max_len": self.max_len, "vocab_size": self.vocab_size,
                   "embed_dim": self.embed_dim})
    return config
```

Sin `get_config()`, un modelo guardado con `model.save` no puede volver a cargarse con sus hiperparámetros.

3. Más capacidad

Más épocas, ya que la validación seguía mejorando; embeddings de más de 16 dimensiones; o varios bloques codificadores apilados, algo que la forma constante de la salida permite.

PARTE VI

Para consolidar

Preguntas para comprobar que se entendió el fondo, un glosario y las conclusiones.

23. Preguntas de autoevaluación

Conviene intentar responder antes de leer la respuesta.

1. ¿Por qué se rellena a `max_len - 1` y no a `max_len`?

Porque el `[CLS]` cuenta dentro de las 200 posiciones: 199 para palabras y 1 para el `[CLS]`.

2. ¿Qué pasaría si se antepusiera el `[CLS]` y luego se recortara por el inicio?

En todas las noticias de más de 199 palabras el `[CLS]` se descartaría, y el clasificador leería en la posición 0 una palabra cualquiera.

3. ¿Por qué el `[CLS]` puede tener id 1 sin chocar con ninguna palabra?

Porque `index_from=2` desplaza los índices de las palabras dos lugares y deja libres el 0 y el 1.

4. ¿Por qué se suma un embedding de posición?

La atención no conoce el orden de los tokens; sin la posición, dos noticias con las mismas palabras en distinto orden serían idénticas para el modelo.

5. ¿Qué hace `x[:, 0, :]` y por qué basta con esa fila?

Toma la salida del `[CLS]`. Gracias a la auto-atención, esa fila mezcló información de toda la noticia y funciona como resumen aprendido.

6. ¿Por qué `sparse_categorical_crossentropy`?

Porque las etiquetas son enteros de 0 a 45. Con etiquetas *one-hot* sería `categorical_crossentropy`.

7. ¿Dónde están casi todos los parámetros del modelo?

En el embedding de palabras, 320.000 de 327.802. El bloque Transformer tiene 3.296.

8. ¿Qué indica que la validación coincida exactamente con la esperada durante cinco épocas?

Que la arquitectura, el orden de creación de capas y la inicialización son los mismos de la referencia.

9. ¿Por qué la celda de ejemplo imprime un texto sin sentido?

Porque no resta el desplazamiento de `index_from=2` al buscar las palabras en el diccionario.

10. ¿Por qué hay que repetir las pruebas 5_1 y 5_2 después de corregir `seq_preprocess`?

Porque `train_model` llama a `seq_preprocess`: cambiar la función cambia los datos con los que se entrena.

24. Glosario

Término	Definición breve
`[CLS]`	Token artificial al inicio de cada secuencia cuya salida resume la noticia; aquí id 1
Relleno o padding	Ceros que completan las noticias cortas, aquí al final
Recorte o truncating	Descartar parte de una noticia larga; aquí el inicio
Embedding	Tabla entrenable que convierte un id en un vector
Embedding de posición	Vector aprendido por cada lugar de la secuencia
Auto-atención	Cada token construye su representación mezclando las de todos los demás
Cabeza de atención	Una de las copias paralelas de la atención, con matrices propias
Conexión residual	Suma de la entrada a la salida de un subbloque
Normalización de capa	Reescala cada vector a media 0 y varianza 1
Pérdida dispersa	Entropía cruzada que recibe la etiqueta como entero
`get_config()`	Método que devuelve los hiperparámetros de una capa para poder guardarla y recargarla
Línea base	Resultado de una estrategia trivial, para dimensionar el del modelo

25. Conclusiones

- **El problema:** clasificar noticias de Reuters en 46 temas con un Transformer construido desde sus piezas.
- **La solución:**
  - un preprocesamiento con `[CLS]`;
  - un embedding doble de token y posición;
  - un bloque codificador con auto-atención, residuales, normalización y dropout;
  - una cabeza densa que solo lee la fila del `[CLS]`.
- **La intervención:** un solo cambio en `seq_preprocess`: conservar el final de las noticias largas y anteponer el `[CLS]` después de recortar. Llevó dos pruebas de 50 a 100 y se resolvió leyendo la retroalimentación del calificador.
- **El resultado:** 12 de 12 pruebas y envío con 100 %. El modelo llegó a 67,9 % de validación, casi el doble de la línea base de la clase mayoritaria y prácticamente igual a la referencia.
- **Lo que lo limita:** un solo bloque, embeddings de 16 dimensiones y atención sobre el relleno. La validación seguía mejorando en la última época.

Anexo · registro completo de las 10 épocas

Época	Accuracy	Pérdida	Accuracy de validación	Pérdida de validación	Validación esperada	Igual
1	0,3195	2,9635	0,4920	2,5244	0,4920	=
2	0,4963	2,3074	0,5392	2,0730	0,5392	=
3	0,5532	1,8922	0,5748	1,7553	0,5748	=
4	0,5941	1,6550	0,6051	1,6448	0,6051	=
5	0,6328	1,5320	0,6402	1,5449	0,6402	=
6	0,6650	1,4291	0,6532	1,4920	0,6523	
7	0,6910	1,3291	0,6705	1,4318	0,6701	
8	0,7110	1,2399	0,6714	1,3826	0,6705	
9	0,7209	1,1683	0,6763	1,3534	0,6763	=
10	0,7341	1,0972	0,6794	1,3299	0,6808	

Créditos del notebook original: profesor Fabio Augusto González, asistente docente Juan Sebastián Malagón Torres y coordinador de virtualización Edder Hernández Forero. Universidad Nacional de Colombia, Facultad de Ingeniería.